

TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

ESTRUTURAS DE DADOS

Organização, Eficiência e Java

Collections

Prof. Eduardo Alencar

Conteúdos de Hoje

- ▶ Conceito e Importância para POO
- ▶ Estruturas de Memória: Vetores
- ▶ Dinamismo com Listas
- ▶ Tipagem com Enums
- ▶ Entendendo Objetos e Nós
- ▶ Estruturas Sequenciais: Pilhas e Filas
- ▶ Hierarquia com Árvores
- ▶ Linear vs. Não-Linear
- ▶ Sintaxe Java (Collections Framework)
- ▶ Exemplos Práticos e Contextualizados

O QUE SÃO ESTRUTURAS DE DADOS?

Conceito e Definição

Estruturas de Dados são formas organizadas de **armazenar, gerenciar e manipular** dados no computador para que possam ser usados de forma eficiente.

Na Programação Orientada a Objetos, elas não apenas guardam valores, mas gerenciam **coleções de objetos**, permitindo que o sistema escale com performance.



Sem estruturas, os dados seriam um amontoado confuso na memória RAM, tornando a busca e a organização impossíveis.

A Importância no Desenvolvimento

- **Gestão de Objetos:** Em Java, quase tudo é um objeto. Estruturas como `ArrayList` permitem manipular milhares de instâncias de forma simples.
- **Desempenho:** Escolher a estrutura correta (ex: `HashSet` vs `ArrayList`) pode reduzir o tempo de execução de minutos para milissegundos.
- **Java Collections Framework:** Java oferece uma biblioteca pronta de estruturas padronizadas, seguras e testadas.
- **Abstração:** Você foca na lógica do negócio, enquanto o Java cuida do gerenciamento de memória.

VETOR (ARRAY)

Estruturas Estáticas

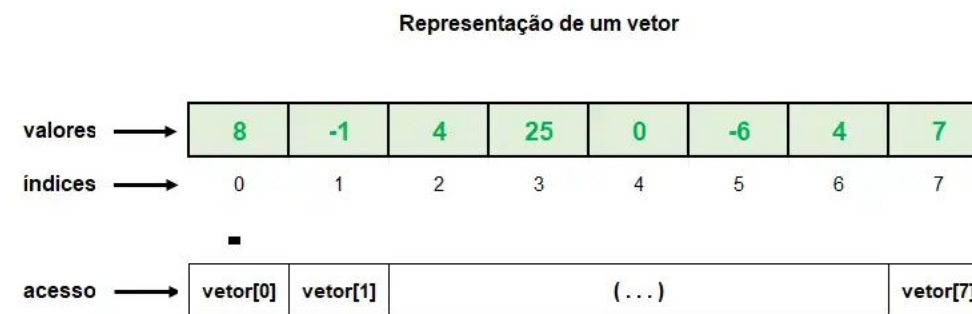
O **Vetor** é a estrutura mais básica: um bloco contíguo de memória com tamanho fixo.

Acesso Direto: Usa índices (começando em 0).

Tamanho Fixo: Não cresce após criado.

Contexto: Ideal para guardar dados onde a quantidade é conhecida (ex: Notas de um bimestre, Dias da semana).

Memória
8
-1
4
25
0
-6
4
7



Sintaxe e Implementação

```
// Declaração de um vetor de inteiros com 5 posições
int[] notas = new int[5];

// Atribuindo valores via índice
notas[0] = 10;
notas[1] = 8;

// Percorrendo o vetor
for (int i = 0; i < notas.length; i++) {
    System.out.println("Nota: " + notas[i]);
}
```

Estruturas Dinâmicas

Diferente do vetor, a **Lista** pode crescer e diminuir conforme a necessidade do programa.



Flexibilidade: Adicione ou remova itens a qualquer momento.

Gestão Automática: O Java cuida da redimensionamento interno.

Contexto: Carrinho de compras de um E-commerce, onde o usuário adiciona quantos itens quiser.

LISTA: SINTAXE GENERICS <>

```
import java.util.ArrayList;  
import java.util.List;  
  
// Definimos o tipo de objeto entre < >  
List<String> nomes = new ArrayList<>();
```

*Dica: Generics garantem que a lista aceite apenas o tipo especificado.

LISTA: MÉTODO .ADD()

Adiciona elementos ao final da lista de forma dinâmica.

```
nomes.add("Eduardo");  
nomes.add("Ana");  
  
// Também permite inserir em posição específica:  
nomes.add(0, "Início");
```

LISTA: MÉTODO .REMOVE()

Remove elementos por índice ou pelo próprio objeto.

```
nomes.remove(0); // Remove quem está no índice 0  
nomes.remove("Ana"); // Procura e remove o objeto "Ana"  
  
// Para apagar TUDO:  
nomes.clear();
```

LISTA: MÉTODO .GET()

Recupera o valor armazenado em um determinado índice.

```
String primeiro = nomes.get(0);  
  
// Uso comum em loops  
for (int i=0; i < nomes.size(); i++) {  
    System.out.println(nomes.get(i));  
}
```

LISTA: MÉTODO .SET()

Substitui o valor em uma posição já existente.

```
// nomes.set(indice, novoObjeto)  
nomes.set(1, "Novo Nome");
```

*Nota: Se o índice não existir, causará erro.

LISTA: MÉTODO .SIZE()

Diferente do .length do vetor, aqui usamos um método para saber a quantidade de itens.

```
int total = nomes.size();  
  
// Verifica se está vazia  
if (nomes.isEmpty()) { ... }
```

LISTA: MÉTODO .CONTAINS()

Retorna um booleano indicando se o objeto está presente na lista.

```
if (nomes.contains("Eduardo")) {  
    System.out.println("O professor está na lista!");  
}
```

LISTA: RESUMO DE COMANDOS

Ação	Comando	O que faz
Inserir	<code>.add(e)</code>	Adiciona no fim
Ler	<code>.get(i)</code>	Pega no índice i
Remover	<code>.remove(i)</code>	Apaga item em i
Alterar	<code>.set(i, e)</code>	Muda item em i
Tamanho	<code>.size()</code>	Qtd de itens atual

ArrayList e Generics

```
import java.util.ArrayList;
import java.util.List;

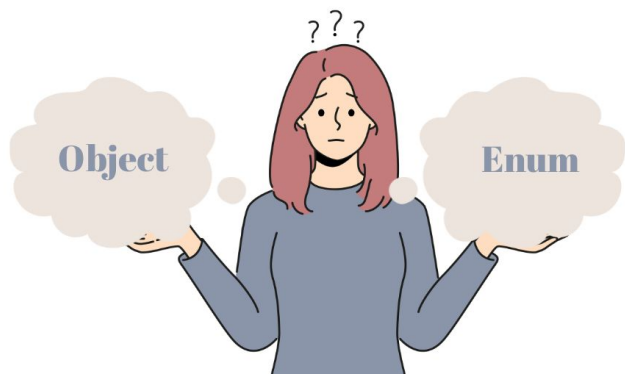
// Criando uma lista de objetos String
List<String> nomes = new ArrayList<>();

nomes.add("Eduardo"); // Adiciona item
nomes.add("Ana");
nomes.remove(0); // Remove por índice

System.out.println("Tamanho: " + nomes.size());
```


Tipos de Dados Constantes

O **Enum** é uma estrutura que define um conjunto fixo de constantes nomeadas.



Evita erros de digitação e garante que o dado seja apenas um dos valores permitidos.

Segurança de Tipo: O compilador garante que você não use um valor inválido.

Semântica: Deixa o código muito mais legível.

Contexto: Dias da semana, Status de um pedido (PENDENTE, PAGO, CANCELADO), Meses do ano.

Definição e Uso

```
public enum StatusPedido {  
    AGUARDANDO_PAGAMENTO, PROCESSANDO, ENVIADO, ENTREGUE;  
}  
  
// No código principal:  
StatusPedido atual = StatusPedido.PROCESSANDO;  
  
if (atual == StatusPedido.ENTREGUE) {  
    System.out.println("Pedido Finalizado");  
}
```

CONCEITO FUNDAMENTAL: NÓS (NODES)

A Base das Estruturas Encadeadas

Para entender Pilhas, Filas e Árvores, precisamos do conceito de **Nó (Node)**.

Um Nó é um objeto que contém duas partes:

Valor/Dado: O objeto real que queremos guardar.

Referência (Ponteiro): O endereço de memória do próximo Nó.



Imagine uma corrente: cada elo guarda sua informação e segura o próximo elo. Se você soltar um elo, a corrente se divide.

LIFO: Last-In, First-Out

O último elemento a entrar é, obrigatoriamente, o primeiro a sair.



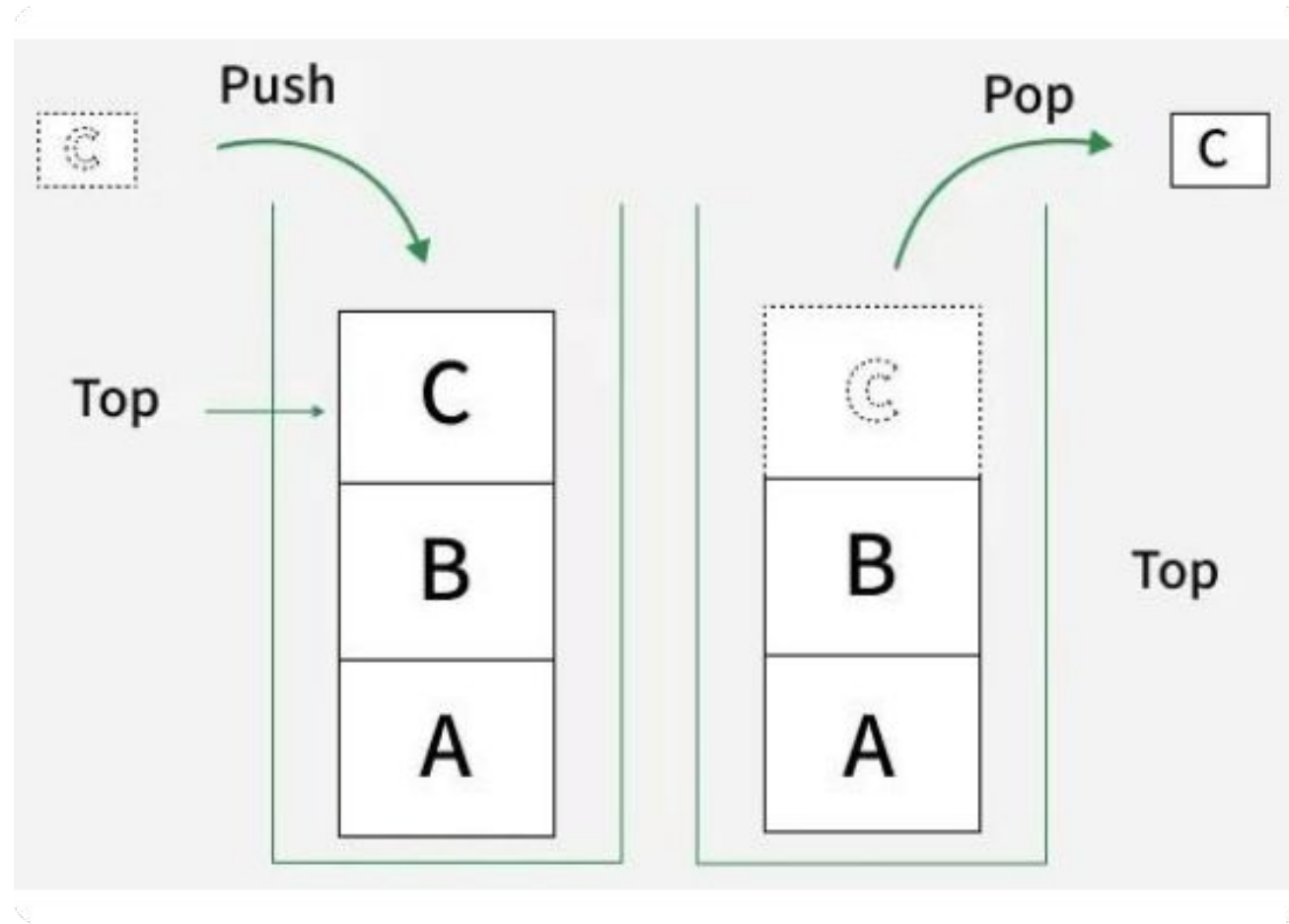
Pense em uma pilha de pratos ou no botão "Desfazer" (Ctrl+Z) do seu editor de texto.

Push: Adiciona ao topo.

Pop: Remove do topo.

Contexto: Histórico de navegação do browser (o último site visitado é o primeiro que aparece ao clicar em 'voltar').

PILHA: REPRESENTAÇÃO



PILHA: SINTAXE GENERICS <>

```
Stack<String> pilha = new Stack<>();
```

*Dica: Generics garantem que a pilha aceite apenas o tipo especificado.

MÉTODO .PUSH()

O método push é utilizado para adicionar um elemento no **topo** da pilha.

```
pilha.push("Primeiro");  
pilha.push("Segundo");  
pilha.push("Terceiro"); // "Terceiro" é o novo topo
```

MÉTODO .PEEK()

Permite observar o elemento que está no topo sem removê-lo da estrutura.

```
// Apenas visualiza o topo atual
System.out.println("Topo: " + pilha.peek());

// Saída: Topo: Terceiro
```


MÉTODO .POP()

Remove e retorna o elemento localizado no topo da pilha.

```
// Remove "Terceiro" e o retorna para a variável  
String removido = pilha.pop();  
  
System.out.println("Removido: " + removido);  
// O elemento "Segundo" agora passa a ser o novo topo.
```

MÉTODO .SIZE()

Retorna o número total de elementos presentes na pilha no momento da execução.

```
// Verifica quantos itens restaram na pilha  
int total = pilha.size();  
  
System.out.println("Total: " + total);
```

MÉTODO .SEARCH()

Busca um objeto e retorna sua distância em relação ao topo (base 1).

```
// Procura o item "Primeiro" na pilha
int posicao = pilha.search("Primeiro");

System.out.println("Posição do 'Primeiro': " + posicao);
// Nota: 0 topo é a posição 1. Se não encontrado, retorna -1.
```

MÉTODO .EMPTY()

Verifica se a pilha está vazia, retornando um valor booleano (true ou false).

```
// Checa se ainda há elementos  
boolean status = pilha.empty();  
  
System.out.println("Está vazia? " + status);
```

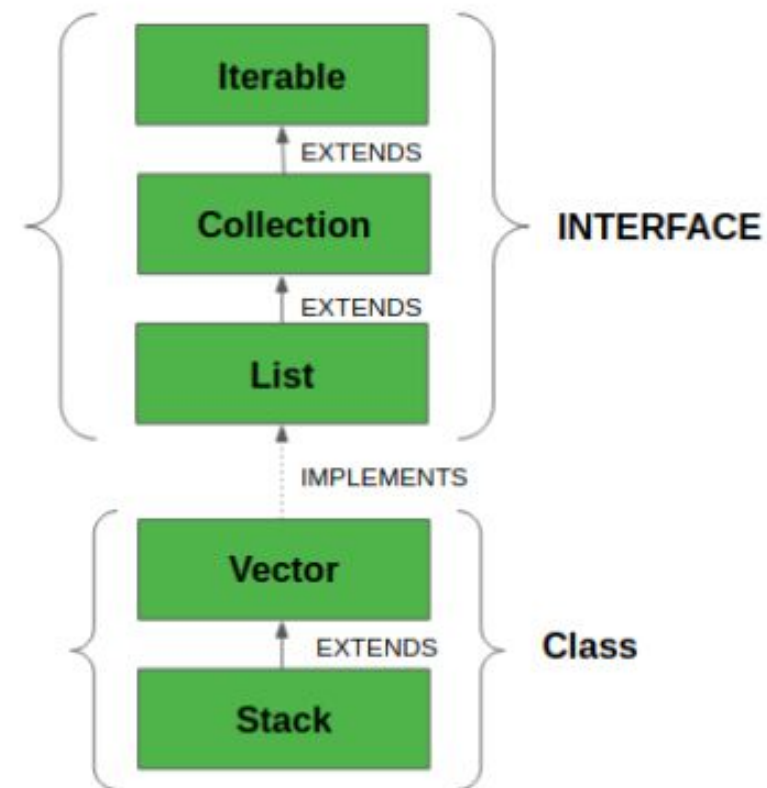
POR QUE PODEMOS PERCORRER?

Em Java, a classe Stack herda da classe Vector.

Vector implementa a interface List.

Isso torna a Pilha uma estrutura híbrida: ela é LIFO, mas também é uma Lista Indexada.

Diferente de pilhas teóricas puras, no Java podemos acessar qualquer posição via índice



VARREDURA COMO VETOR

Como herda de Vector, podemos utilizar o método `.get(i)` para acessar elementos pela ordem de entrada (índice 0 é a base).

```
for (int i = 0; i < pilha.size(); i++) { // Acesso indexado (0-based da base para o top
```

ORDEM DE INCLUSÃO NA PILHA

O método `search()` conta do topo para a base. Para saber a ordem real de inserção (de 1 a N), usamos a lógica:

$$\text{OrdemInsercao} = \text{size()} - \text{search(elemento)} - 1$$

```
// Exemplo prático:  
int ordem = pilha.size() - (pilha.search("Primeiro") - 1);  
System.out.println("Foi o " + ordem + "º a ser inserido.");
```

FIFO: First-In, First-Out

O primeiro elemento a entrar é o primeiro a ser processado e sair.

Enqueue: Entra no **fim** da fila.

Dequeue: Sai do **início** da fila.

Ordem: Preserva a sequência cronológica de chegada

Contexto: Fila de impressão de documentos ou sistema de atendimento por senhas.



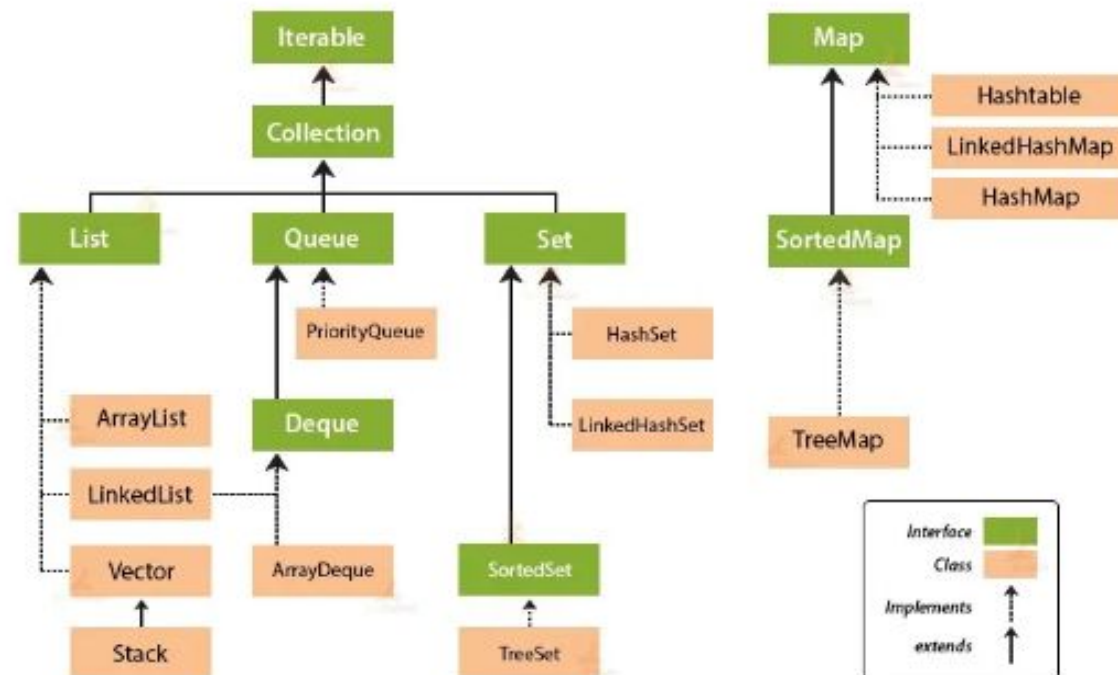
LINKEDLIST E PRIORITYQUEUE

A interface Queue não pode ser instanciada diretamente. Usamos classes que a implementam:

LinkedList: A mais comum. Fila padrão FIFO simples.

PriorityQueue: Os itens são ordenados por prioridade (natural ou via Comparator), não apenas chegada.

Collection Framework Hierarchy in Java



FILA: SINTAXE E GENERICS <>

```
import java.util.Queue; import java.util.LinkedList;
// Queue é uma Interface; LinkedList é a implementação
Queue fila = new LinkedList<>();
```

***Dica:** Como **Queue** é uma interface no Java Collections Framework, precisamos instanciar uma classe concreta como **LinkedList** ou **PriorityQueue**. O Generics garante que todos os elementos na fila sejam do mesmo tipo.



ADD(E) VS OFFER(E)

add(e)

Tenta inserir. Se a fila tiver limite de capacidade e estiver cheia, lança `IllegalStateException`.

```
fila.add("João");
```

offer(e)

Tenta inserir. Se estiver cheia, apenas retorna **false** em vez de quebrar o programa.

```
fila.offer("Maria");
```

REMOVE() VS POLL()

remove()

Remove o primeiro da fila. Se a fila estiver vazia, lança `NoSuchElementException`.

```
String s = fila.remove();
```

poll()

Remove o primeiro. Se vazia, retorna **null**. Muito mais seguro para loops.

```
String s = fila.poll();
```

ELEMENT() VS PEEK()

element()

Retorna o primeiro da fila sem remover. Erro
NoSuchElementException se vazia.

```
System.out.println(fila.element());
```

peek()

Apenas "espia" o primeiro. Retorna **null** se não
houver ninguém na fila.

```
System.out.println(fila.peek());
```

UTILIDADES IMPORTANTES



.size()

Retorna a quantidade de elementos atuais.



.isEmpty()

Verifica se a fila não possui nenhum item.



.contains(obj)

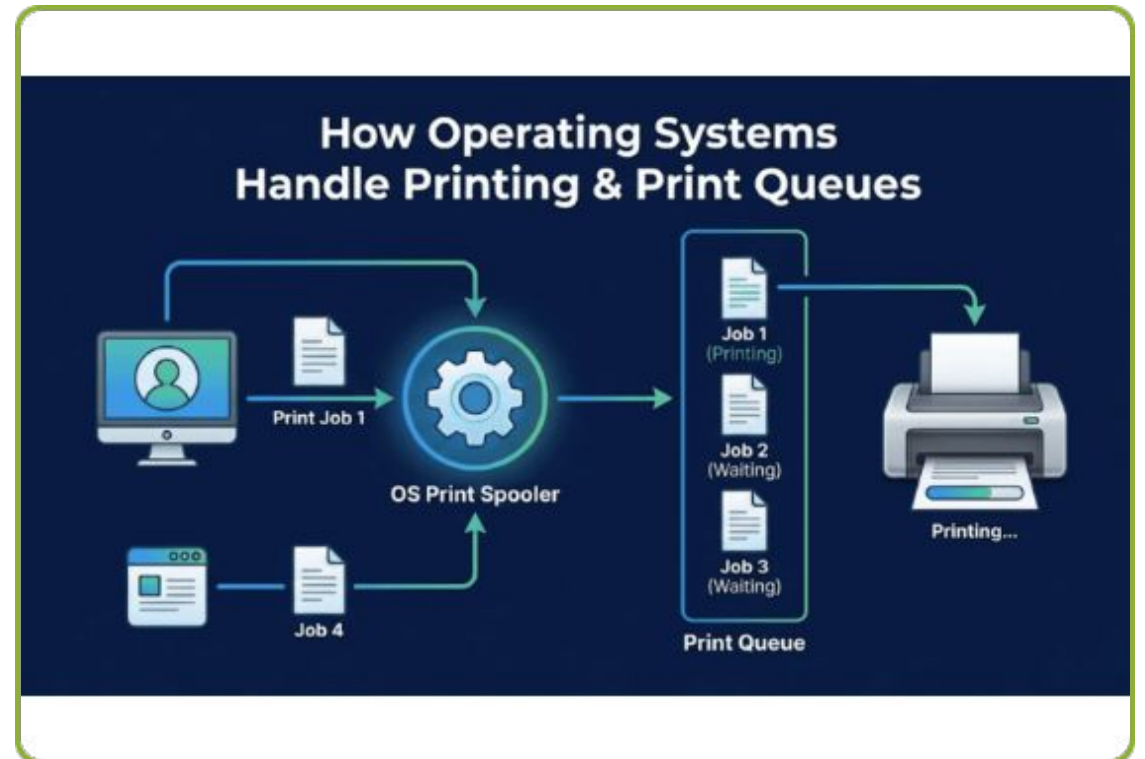
Busca se um objeto específico está na fila.

```
if (!fila.isEmpty()) { System.out.println("Tamanho atual: " + fila.size()); }
```

FILA DE IMPRESSÃO

Imagine um sistema que gerencia documentos para uma impressora:

```
Queue impressora = new LinkedList<>();  
impressora.offer("Trabalho_TI.pdf");  
impressora.offer("Relatorio_Final.docx"); while  
(!impressora.isEmpty()) {  
    System.out.println("Imprimindo: " +  
    impressora.poll()); }
```



ITERANDO SOBRE A FILA

Embora a fila seja feita para remover o primeiro, às vezes precisamos listar todos os itens sem destruí-la:

For-Each (Simples)

```
for (String pessoa : fila) {  
    System.out.println(pessoa);  
}
```

Iterator (Controle)

```
Iterator it = fila.iterator(); while  
(it.hasNext()) {  
    System.out.println(it.next());  
}
```


RESUMO DE MÉTODOS

Operação	Lança Exceção	Retorna Valor Especial
Inserir	add(e)	offer(e)
Remover	remove()	poll()
Examinar	element()	peek()
Tamanho	#	size()
Verificar Vazio		isEmpty()
Buscar Elemento	casos específicos*	contains(obj)*

Dica: Prefira sempre a coluna da direita para evitar erros de tempo de execução (Runtime Errors).

MÉTODOS ÚTEIS DE MANIPULAÇÃO

.clear()

Remove todos os elementos da fila de uma vez. Essencial para resetar processos ou limpar buffers de memória.

```
fila.clear();
```

.addAll(c)

Adiciona todos os elementos de uma coleção (List, Set) ao final da fila. Útil para carregamento em lote.

```
fila.addAll(listaDeDados);
```

.toArray()

Converte a fila em um vetor estático (Array). Facilita o envio de dados para APIs que não aceitam Coleções.

```
Object[] vetor = fila.toArray();
```

.removeIf(p)

Remove elementos que atendem a uma condição (Predicado). Útil para "limpar" itens expirados ou inválidos.

```
fila.removeIf(x → x.equals(null));
```

Lineares vs. Não-Lineares

Estruturas Lineares

Os elementos são organizados em sequência (um após o outro).

Ex: Vetores, Listas, Pilhas, Filas.

Uso: Processamento sequencial e armazenamento simples.

Estruturas Não-Lineares

Os elementos são organizados de forma hierárquica ou em rede.

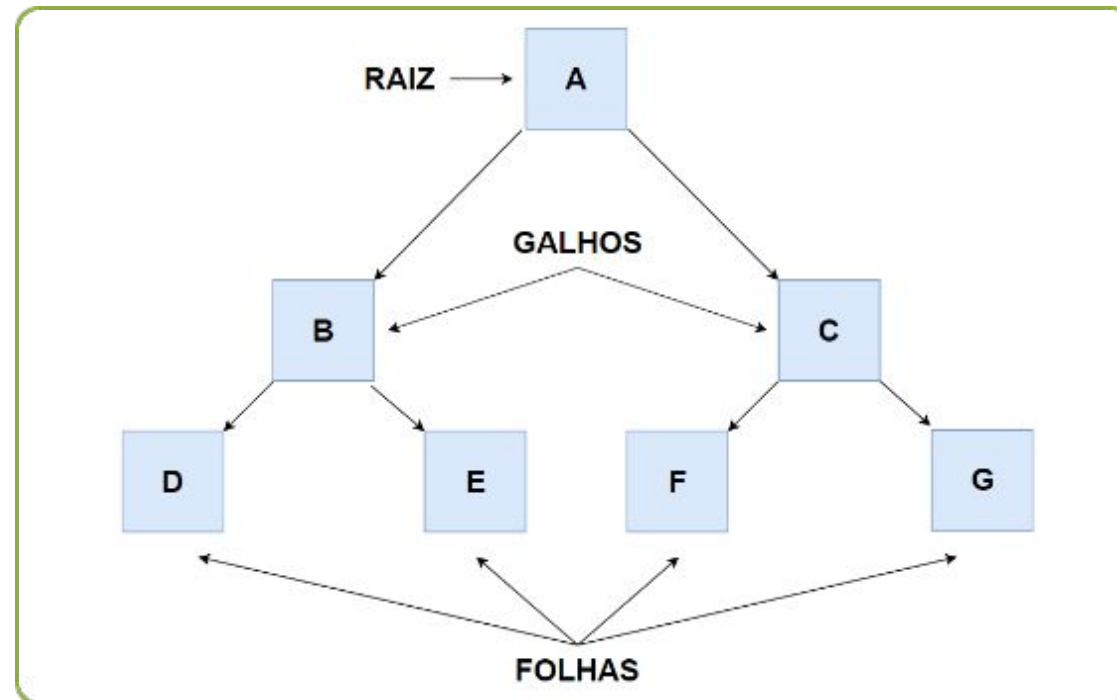
Ex: Árvores, Grafos.

Uso: Mapeamento de relações complexas e buscas rápidas.

ESTRUTURA NÃO-LINEAR

Diferente de Listas, Pilhas e Filas, a Árvore organiza os dados de forma **hierárquica**.

- **Não-Sequencial:** Um elemento pode ter múltiplos sucessores.
- **Relação Pai-Filho:** Conexões que definem níveis de profundidade.
- **Eficiência:** Permite buscas, inserções e remoções rápidas (logarítmicas).
- **Contexto:** Sistema de arquivos do Windows (Pastas), Organograma de uma empresa ou busca binária otimizada.



ANATOMIA: O CONCEITO DE NÓS

Toda árvore é composta por **Nós** (Nodes), que são os recipientes dos dados.

- **Raiz (Root):** O nó principal, no topo da hierarquia.
- **Filho (Child):** Nó descendente de outro nó.
- **Pai (Parent):** Nó que possui ramificações abaixo dele.
- **Folha (Leaf):** Nó que não possui filhos (fim da linha).

Subárvore: Qualquer nó, juntamente com todos os seus descendentes, forma uma subárvore.

INSTANCIACÃO E GENERICS

No Java Collections, a implementação mais comum é o **TreeMap**, que mantém os elementos ordenados por suas chaves.

```
import java.util.TreeMap; // Chave (Integer) e Valor (String) TreeMap arvore = new TreeMap<>();
```

*Nota: TreeMap utiliza uma Árvore Rubro-Negra (Red-Black Tree) internamente para garantir o balanceamento.

MÉTODO .PUT(K, V)

Conceito

Insere um novo nó na árvore associando uma chave a um valor. Se a chave já existir, o valor é atualizado.

```
// Inserindo elementos na árvore arvore.put(10, "Raiz"); arvore.put(5, "Filho Esquerdo");  
arvore.put(15, "Filho Direito");
```

MÉTODO .GET(K)

Conceito

Busca e retorna o valor associado a uma chave específica. Se a chave não existir, retorna **null**.

```
// Recuperando o valor da chave 5 String valor = arvore.get(5); System.out.println("Dado encontrado: " + valor);
```


MÉTODO .CONTAINSKEY(K)

Conceito

Verifica se uma determinada chave está presente na estrutura. Retorna um booleano.

```
if (arvore.containsKey(15)) { System.out.println("O nó com chave 15 existe!"); }
```

FIRSTKEY() E LASTKEY()

Conceito

Como a árvore é ordenada, esses métodos retornam a menor (mais à esquerda) e a maior (mais à direita) chave da árvore.

```
System.out.println("Menor chave: " + arvore.firstKey()); System.out.println("Maior  
chave: " + arvore.lastKey());
```

MÉTODO .REMOVE(K)

Conceito

Remove o nó correspondente à chave informada e reorganiza a estrutura para manter a hierarquia.

```
// Remove o elemento e retorna seu valor String removido = arvore.remove(10);  
System.out.println("Nó removido: " + removido);
```

MÉTODO .SIZE()

Conceito

Retorna o número total de nós (elementos) atualmente armazenados na árvore.

```
int totalNo = arvore.size(); System.out.println("Total de nós: " + totalNo);
```

TABELA DE MÉTODOS TRABALHADOS

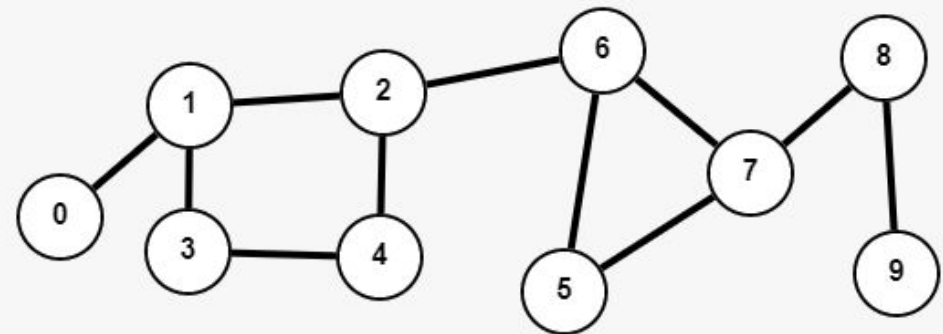
Método	Ação Principal	Retorno Esperado
put(K, V)	Inserir ou atualizar um nó	V (valor antigo ou null)
get(K)	Busca valor pela chave	V (valor ou null)
containsKey(K)	Verifica existência da chave	boolean
firstKey() / lastKey()	Busca limites da árvore	K (chave)
remove(K)	Exclui um nó específico	V (valor removido)
size()	Contagem de elementos	int

O QUE É UM GRAFO?

O Grafo é a estrutura **Não-Linear** mais genérica que existe. Ele representa um conjunto de objetos e as conexões entre eles.

Interconectividade: Ao contrário das árvores, grafos podem ter ciclos (caminhos que voltam ao início).

Aplicações: Redes sociais (amizades), Google Maps (rotas), Redes de computadores e Internet.



VÉRTICES E ARESTAS

Um grafo $G = (V, A)$ é definido por dois componentes fundamentais:

Vértices (V)

Também chamados de **Nós**. Representam as entidades (Pessoas, Cidades, Servidores).

Arestas (A)

Representam as **Conexões** ou relacionamentos entre os vértices.

Grau: O número de arestas conectadas a um vértice.

LISTA DE ADJACÊNCIA

Como o Java não possui uma classe Graph nativa, usamos o conceito de **Lista de Adjacência** com Map e List.

```
import java.util.*; // Cada Vértice (String) aponta para uma lista de vizinhos
Map<String, List<String>> grafo = new HashMap<>();
```

Essa é a forma mais eficiente em memória para grafos com muitas entidades e poucas conexões (esparsos).

ADICIONAR VÉRTICE

Conceito

Cria uma nova entidade no grafo e inicializa sua lista de conexões vazia.

```
public void adicionarVertice(String nome) { // Cria o nó e uma lista vazia de vizinhos  
    grafo.putIfAbsent(nome, new ArrayList<>()); }
```

ADICIONAR ARESTA

Conceito

Estabelece um vínculo entre dois vértices existentes. Em grafos não-direcionados, a conexão é mútua.

```
public void adicionarAresta(String origem, String destino) { // Conecta origem ao destino
    grafo.get(origem).add(destino); // Conecta destino à origem (Não-Direcionado)
    grafo.get(destino).add(origem); }
```

POSSUI ARESTA?

Conceito

Verifica se existe uma conexão direta entre dois vértices específicos.

```
public boolean possuiConexao(String v1, String v2) { // Verifica se v2 está na lista de vizinhos de v1
return grafo.containsKey(v1) && grafo.get(v1).contains(v2); }
```

OBTER VIZINHOS

Conceito

Retorna a lista de todos os vértices que possuem uma conexão direta com o nó informado.

```
public List vizinhosDe(String nome) { // Retorna a lista de adjacência do nó return  
    grafo.getDefault(nome, Collections.emptyList()); }
```

REMOVER ARESTA

Conceito

Desfaz o vínculo entre dois nós, removendo a referência de um na lista do outro.

```
public void removerAresta(String v1, String v2) { if (grafo.containsKey(v1))  
    grafo.get(v1).remove(v2); if (grafo.containsKey(v2)) grafo.get(v2).remove(v1); }
```

CONTAR VÉRTICES (TAMANHO)

Conceito

Informa o número total de entidades mapeadas no grafo.

```
public int totalVertices() { // O tamanho do Map representa o total de nós return grafo.size(); }
```

TABELA DE MÉTODOS DE GRAFO

Operação	Método Proposto	Objetivo Didático
Inserir Nó	adicionarVertice(V)	Criar a entidade no Map
Inserir Vínculo	adicionarAresta(V, V)	Preencher as listas de adjacência
Verificar	possuiConexao(V, V)	Checar adjacência direta
Listar	vizinhosDe(V)	Percorrer ramificações locais
Excluir Vínculo	removerAresta(V, V)	Romper conexão entre nós
Mensurar	totalVertices()	Obter a ordem do grafo

Qual Estrutura Escolher?

Necessidade	Estrutura Recomendada	Por quê?
Tamanho fixo, alta performance	Vetor (Array)	Acesso por índice é instantâneo.
Lista que cresce/diminui	ArrayList / LinkedList	Dinâmica e fácil manipulação.
Inverter ordem ou Desfazer	Pilha (Stack)	Lógica LIFO natural.
Atendimento por ordem de chegada / Buffer de dados	Fila (Queue / LinkedList)	Garante a lógica FIFO (First-In, First-Out), processando os elementos estritamente na ordem cronológica de entrada.
Organização Hierárquica	Árvore (Tree)	Relação pai-filho e busca eficiente.
Redes de interconexão, mapas ou relacionamentos complexos	Grafo (Graph / Map + List)	Modela relações muitos-para-muitos não-lineares, permitindo ciclos, caminhos e pesos entre nós.



Dúvidas?

Aula: Estruturas de Dados em Java

Email: eduardo.junior@ifb.edu.br